URL Shortener Application Deployment on Amazon EKS with GitOps and CI/CD

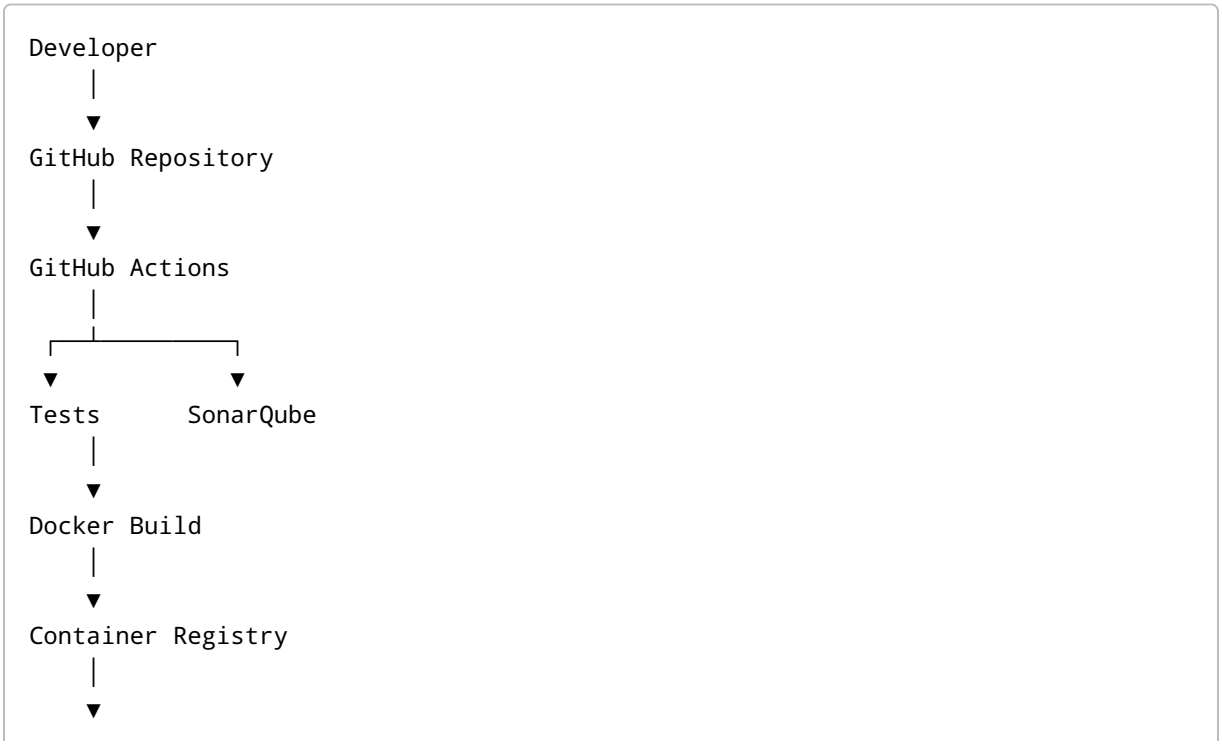
Project Overview

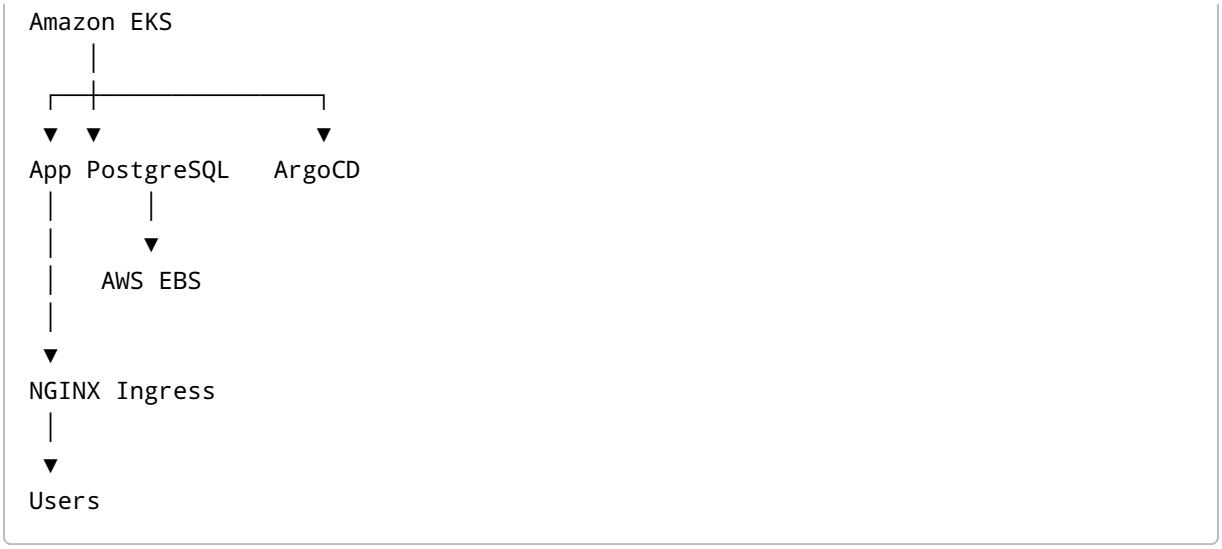
This project demonstrates the end-to-end deployment of a containerized Django-based URL Shortener application on Amazon EKS (Elastic Kubernetes Service).

The project covers:

- Docker containerization
- Kubernetes deployments
- Amazon EKS cluster administration
- PostgreSQL database deployment
- Persistent storage using AWS EBS
- NGINX Ingress Controller
- ArgoCD GitOps deployment
- GitHub Actions CI pipeline
- SonarQube code quality integration

Architecture





Technology Stack

Component	Technology
Cloud	AWS
Containerization	Docker
Orchestration	Kubernetes
Managed Kubernetes	Amazon EKS
Database	PostgreSQL
Storage	AWS EBS
Ingress	NGINX Ingress Controller
GitOps	ArgoCD
CI/CD	GitHub Actions
Code Quality	SonarQube
Language	Python
Framework	Django

Prerequisites

Before starting:

- AWS Account
- IAM User with EKS permissions
- AWS CLI configured
- kubectl installed
- eksctl installed
- Docker installed
- Git installed
- Helm installed

Step 1: Containerize the Application

Created a Dockerfile for the Django application.

Build image:

```
docker build -t url-shortener .
```

Run locally:

```
docker run -p 8000:8000 url-shortener
```

Verify application functionality before deployment.

Step 2: Push Image to Registry

Login:

```
docker login
```

Tag image:

```
docker tag url-shortener username/url-shortener:v1
```

Push image:

```
docker push username/url-shortener:v1
```

Step 3: Create Amazon EKS Cluster

Provisioned an EKS cluster.

Configure kubectl:

```
aws eks update-kubeconfig
--region ap-south-1
--name url-shortener-cluster
```

Verify nodes:

```
kubectl get nodes
```

Expected:

```
STATUS: Ready
```

Step 4: Create Kubernetes Namespace

```
apiVersion: v1
kind: Namespace
metadata:
  name: url-shortener
```

Apply:

```
kubectl apply -f namespace.yaml
```

Step 5: Configure Image Pull Secret

Required for pulling private container images.

```
kubectl create secret docker-registry regcred
--docker-username=<username>
--docker-password=<password>
--docker-email=<email>
-n url-shortener
```

Verify:

```
kubectl get secrets -n url-shortener
```

Step 6: Deploy PostgreSQL

Created:

- Deployment
- Service
- PersistentVolumeClaim

Database configuration:

```
POSTGRES_DB
POSTGRES_USER
POSTGRES_PASSWORD
```

Verify:

```
kubectl get pods -n url-shortener
```

Step 7: Configure Persistent Storage

Amazon EKS uses the AWS EBS CSI Driver for dynamic volume provisioning.

Verify available Storage Classes:

```
kubectl get storageclass
```

Example:

```
gp3 (default)
```

Create PVC:

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: postgres-pvc
spec:
  storageClassName: gp3

  accessModes:
    - ReadWriteOnce

  resources:
    requests:
      storage: 10Gi
```

When the PVC is created:

```
PVC
 ↓
StorageClass (gp3)
 ↓
AWS EBS Volume
 ↓
PostgreSQL Pod
```

Verify:

```
kubectl get pvc
```

Step 8: Deploy Application

Deployment includes:

- Django container
- Environment variables
- Database configuration
- Image Pull Secret

Apply:

```
kubectl apply -f deployment.yaml
```

Verify:

```
kubectl get deployments
kubectl get pods
```

Step 9: Create Service

Created a ClusterIP service for internal communication.

```
kind: Service
spec:
  selector:
    app: url-shortener
```

Apply:

```
kubectl apply -f service.yaml
```

Verify:

```
kubectl get svc
```

Step 10: Configure Health Checks

Implemented:

Readiness Probe

Ensures application is ready to receive traffic.

Liveness Probe

Detects unhealthy containers and restarts them automatically.

Troubleshooting performed:

- Diagnosed HTTP 405 errors
- Updated probe endpoints
- Verified successful probe responses

Step 11: Install NGINX Ingress Controller

Installed NGINX Ingress Controller.

Verify:

```
kubectl get pods -n ingress-nginx
```

Step 12: Configure Ingress

Created an Ingress resource to expose the application.

Responsibilities:

- External access
- Request routing
- Centralized entry point

Verify:

```
kubectl get ingress
```

Step 13: Deploy and Verify Application

Checked:

```
kubectl get pods
kubectl get svc
kubectl get ingress
```

Verified successful access through the Ingress endpoint.

Step 14: Install ArgoCD

Create namespace:

```
kubectl create namespace argocd
```

Install ArgoCD:

```
kubectl apply -n argocd
-f https://raw.githubusercontent.com/argoproj/argo-cd/stable/manifests/
install.yaml
```

Verify:

```
kubectl get pods -n argocd
```

Step 15: Expose ArgoCD UI

Changed ArgoCD service type to NodePort.

```
kubectl edit svc argocd-server -n argocd
```

Update:

```
type: NodePort
```

Verify:

```
kubectl get svc -n argocd
```

Access:

```
https://<NODE-IP>:<NODEPORT>
```

Step 16: Retrieve ArgoCD Admin Password

```
kubectl -n argocd get secret argocd-initial-admin-secret
-o jsonpath="{.data.password}" | base64 -d
```

Login:

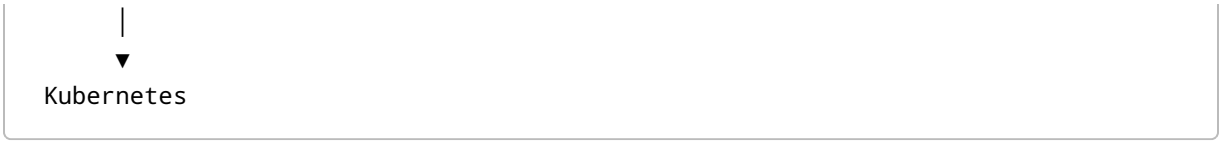
```
Username: admin
Password: <retrieved-password>
```

Step 17: GitOps Workflow

Configured ArgoCD to monitor Kubernetes manifests.

Workflow:

```
Git Repository
      |
      ▼
   ArgoCD
```



Benefits:

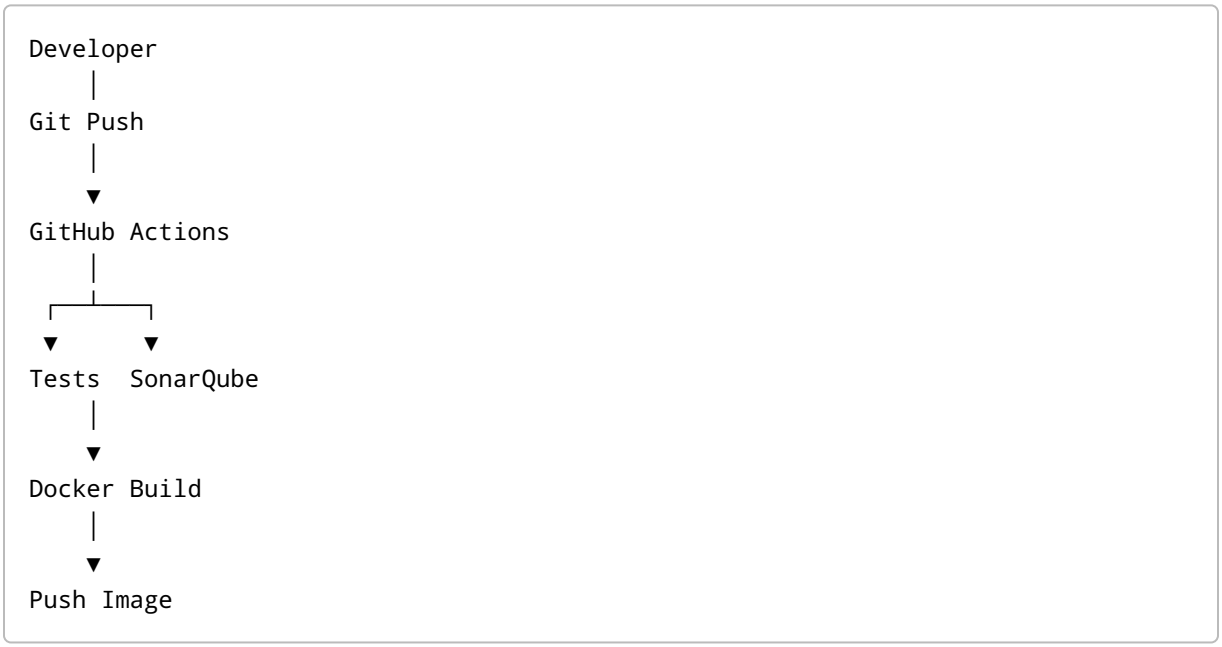
- Declarative deployments
- Automatic synchronization
- Rollback support
- Drift detection

Step 18: GitHub Actions CI Pipeline

Pipeline stages:

1. Checkout Source Code
2. Run Tests
3. SonarQube Analysis
4. Build Docker Image
5. Push Docker Image

Workflow:



Troubleshooting Highlights

Image Pull Errors

```
kubectl describe pod <pod-name>
```

Probe Failures

```
kubectl logs <pod-name>
```

PVC Issues

```
kubectl get pvc
```

Service Connectivity

```
kubectl get svc
```

Ingress Issues

```
kubectl describe ingress
```

Key Learning Outcomes

- Docker image creation and management
- Kubernetes deployments and services
- EKS cluster administration
- Persistent storage using AWS EBS
- Kubernetes health checks
- Ingress configuration
- GitOps implementation using ArgoCD
- CI/CD pipeline design using GitHub Actions
- SonarQube integration
- Kubernetes troubleshooting and debugging

Future Enhancements

- Terraform-based EKS provisioning
- AWS Load Balancer Controller
- ExternalDNS
- cert-manager
- Prometheus monitoring
- Grafana dashboards
- ArgoCD Auto Sync
- Blue/Green deployments
- Canary deployments
- Karpenter autoscaling